



Universidade do Minho

Mestrado em Cibersegurança

Laboratório de Cibersegurança 1

Carlos Daniel Silva Fernandes
(PG59783)

Pedro Augusto Ennes de Martino Camargo
(PG59791)

Luís Filipe Pinheiro Silva
(PG59790)

1 de maio de 2026

Conteúdo

1	Executive Summary	4
2	Scope and Methodology	5
2.1	Pre engagement	5
3	Phase A Findings	6
3.1	SQL Injection in API Endpoint	6
3.1.1	Root Cause	6
3.1.2	Impact Analysis	6
3.1.3	Evidence	6
3.1.4	Remediation Recommendations	7
3.2	Broken Access Control	8
3.2.1	Root Cause	8
3.2.2	Impact Analysis	8
3.2.3	Evidence	8
3.2.4	Remediation Recommendations	9
3.3	2FA Bypass via TOTP Secret Exposure	10
3.3.1	Root Cause	10
3.3.2	Impact Analysis	10
3.3.3	Evidence	10
3.3.4	Remediation Recommendations	11
3.4	JWT Token Forgery	11
3.4.1	Root Cause	12
3.4.2	Impact Analysis	12
3.4.3	Evidence	12
3.4.4	Remediation Recommendations	13
4	Phase B Findings	14
4.1	Attack Plan	14
4.1.1	Brute Force Testing Plan	14
4.1.2	Cross-Site Request Forgery Testing Plan	14
4.1.3	Insecure CAPTCHA Testing Plan	15
4.2	Summary of Findings	15
4.3	Finding B-01: [Finding Title]	15
4.4	Finding B-02: [Finding Title]	17
4.5	Finding B-03: [Finding Title]	19
4.6	Finding B-04: [Finding Title]	21
4.7	Constraint Coverage Matrix	23
4.8	Phase B Conclusion	23
5	Cross-Phase Analysis	24
6	Conclusion	25
7	Appendix A - Evidence	26
7.1	SQL Injection in /rest/products/search	26

Capítulo 1

Executive Summary

High-level summary of scope, key findings, and top recommendations. Written for a non-technical audience. 1 page maximum.

Capítulo 2

Scope and Methodology

Following the PTES framework, this section details the engagement scope, rules of engagement, intelligence gathering, and attack planning processes. It also outlines the tools and techniques used, and any deviations from the standard PTES approach.

2.1 Pre engagement

The applications targeted in this engagement were the **OWASP Juice Shop** and the **Damn Vulnerable Web Application (DVWA)**. Both applications are intentionally vulnerable web applications designed for security training and testing purposes.

In this report, we will document the findings from our engagements with both applications, following the PTES and WSTG frameworks. The report is structured to provide a comprehensive overview of our activities, findings, and recommendations for each phase of the engagement. However, we will not exploit and destroy any systems.

We plan to use tools such as **Burp Suite** for web application testing, **Nmap** for network scanning, and **Metasploit** for exploitation. We will also utilize manual testing techniques to identify vulnerabilities and assess the security posture of the applications.

Furthermore, it is important to note that we will adhere to ethical guidelines and legal requirements throughout the engagement, ensuring that all activities are conducted in a responsible and professional manner.

Capítulo 3

Phase A Findings

3.1 SQL Injection in API Endpoint

Vulnerability Overview	
Vulnerability Class	Injection (SQL Injection)
Location	API Endpoint: /rest/products/search
CVSS v4 Score	9.1 (Critical)
MITRE ATT&CK	Tactic: Initial Access Technique ID: T1190

3.1.1 Root Cause

The API endpoint /rest/products/search constructs SQL queries dynamically using user-supplied input from URL parameters. This is done through string concatenation without proper sanitization or validation.

As a result, an attacker can inject malicious SQL payloads, manipulate query execution, and retrieve or modify unintended data from the SQLite database.

3.1.2 Impact Analysis

This vulnerability critically impacts all three pillars of the CIA triad:

Security Property	Impact
Confidentiality	Unauthorized access to sensitive data via SELECT queries.
Integrity	Data manipulation or corruption through INSERT, UPDATE, or DELETE.
Availability	Malformed queries or exploitation can trigger database errors, leading to service disruption.

3.1.3 Evidence

The vulnerability was confirmed using SQL injection payloads submitted through URL parameters.

```
.../rest/products/search?q=apple')) UNION SELECT id,email,password,4,5,6,7,8,9 FROM users --
```

Listing 3.1: URL Payload

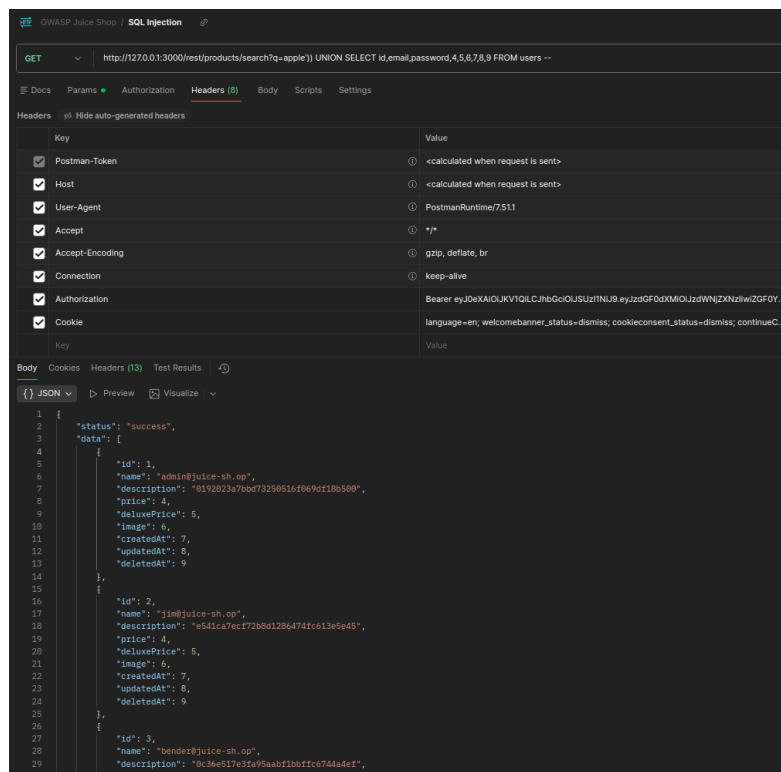


Figura 3.1: SQL injection payload submitted to the `/rest/products/search` endpoint via Postman. The response includes all users names and passwords, demonstrating successful exploitation.

3.1.4 Remediation Recommendations

Recommended Fixes

1. Use Parameterized Queries

Replace dynamic SQL construction with prepared statements or parameterized queries to ensure user input is treated strictly as data.

2. Input Validation

Implement strict validation and sanitization of all user inputs, especially those used in database queries.

3. Secure Error Handling

Avoid exposing internal database errors. Replace verbose messages such as:

```
"error": "syntax error near UNION SELECT..."
```

with generic responses:

```
"error": "Invalid request"
```

4. Principle of Least Privilege

Ensure the database user has minimal required permissions to limit the impact of exploitation.

3.2 Broken Access Control

Vulnerability Overview	
Vulnerability Class	Broken Access Control
Location	API Endpoint: /rest/basket/[Basket-ID]
CVSS v4 Score	5.3 (Medium)
MITRE ATT&CK	Tactic: Initial Access Technique ID: T1078

3.2.1 Root Cause

The server does not validate cookie session and JWT token with userId. If a malicious user connected use the api route to get another basket by changing the ID in the URL, access to data he's not supposed to have is granted.

With this, an attacker has access to other users' baskets by manipulating the Basket-ID in the URL, without proper authorization checks, returning non-sensitive business related information such as **date of creation**, **total price**, and **product details**.

3.2.2 Impact Analysis

This vulnerability primarily impacts confidentiality, as it allows unauthorized access to other users' basket information. While the data exposed may not be highly sensitive, it can still lead to privacy concerns and potential misuse of information.

Security Property	Impact
Confidentiality	Unauthorized access to other users' basket information, including product details and total price.
Integrity	No direct impact on data integrity, as the vulnerability does not allow modification of data.
Availability	No direct impact on availability, as the vulnerability does not cause service disruption.

3.2.3 Evidence

The vulnerability was confirmed by manipulating the Basket-ID in the URL and successfully retrieving another user's basket information without proper authorization.

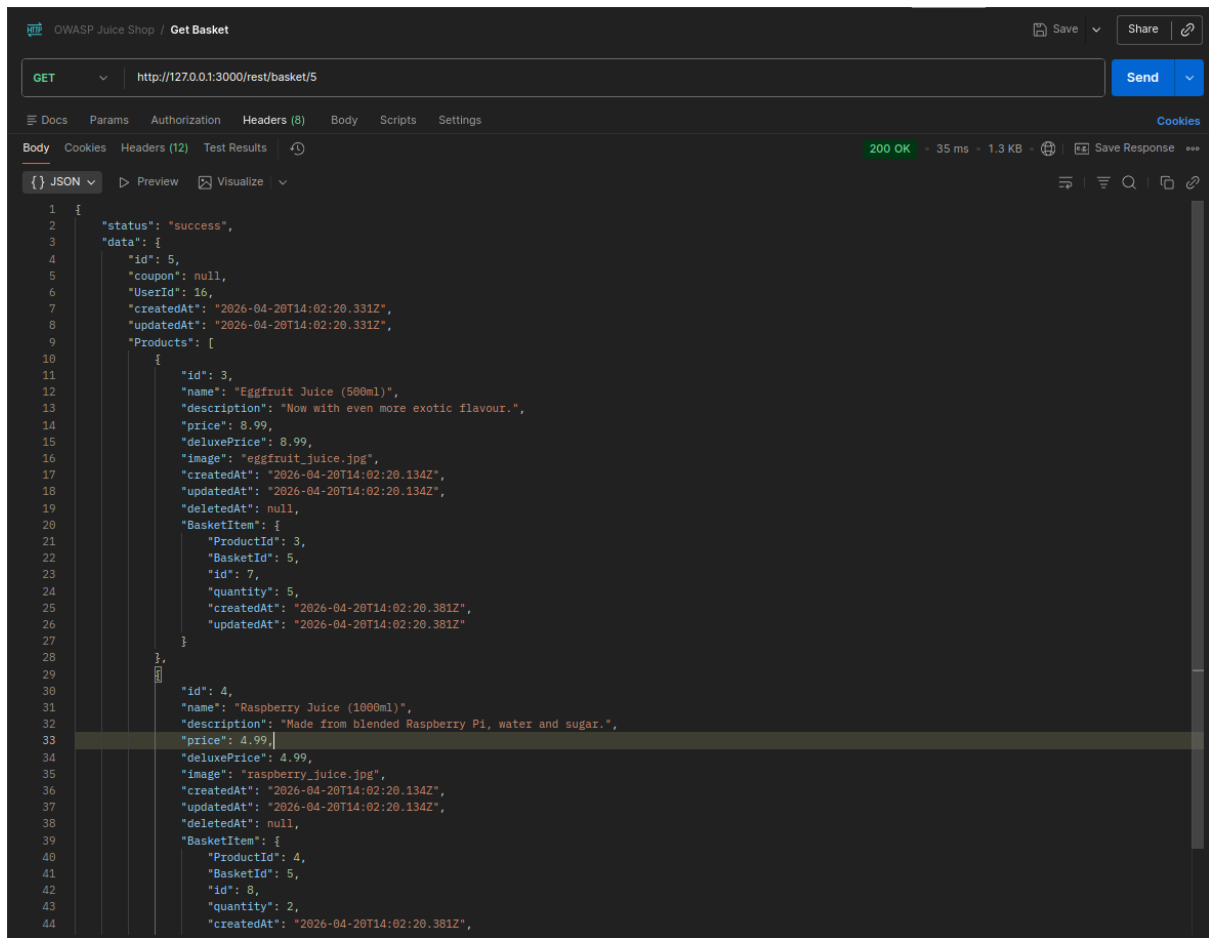


Figure 3.2: Postman request demonstrating broken access control by changing the Basket-ID in the URL to access another user's basket information. (Current user basket id is 6)

3.2.4 Remediation Recommendations

Recommended Fixes

1. Implement Proper Authorization Checks

Ensure that the server validates the user's session and JWT token against the requested Basket-ID to confirm that the user has permission to access that specific resource.

2. Enforce Role-Based Access Control (RBAC)

Implement RBAC to restrict access to resources based on user roles and permissions, ensuring that users can only access their own data.

3. Use Unique Identifiers

Consider using unique identifiers that are not easily guessable for resources like baskets, to make it harder for attackers to enumerate and access other users' data.

3.3 2FA Bypass via TOTP Secret Exposure

Vulnerability Overview	
Vulnerability Class	Authentication
Location	Database and Login Panel
CVSS v4 Score	8.9 (High)
MITRE ATT&CK	Tactic: Credential Access ID: T1555 (Credentials from Password Stores)

3.3.1 Root Cause

Due to the presence of other vulnerabilities in the system, it is possible to extract sensitive user data directly from the database. Among this data is the TOTP secret used for two-factor authentication (2FA), which is stored in plaintext without any form of encryption or protection.

Because the secret is not encrypted, an attacker can easily import it into a standard 2FA application (e.g., Google Authenticator) and generate valid one-time passwords for the victim account.

3.3.2 Impact Analysis

This vulnerability compromises authentication mechanisms and allows full account takeover:

Security Property	Impact
Confidentiality	Exposure of TOTP secrets enables attackers to impersonate users.
Integrity	Unauthorized access allows attackers to modify user data and system state.
Availability	Accounts may be locked or abused, impacting legitimate user access.

3.3.3 Evidence

The TOTP secret was retrieved from the database in plaintext format. The field storing the secret was not protected or encrypted, allowing direct reuse in a 2FA application.

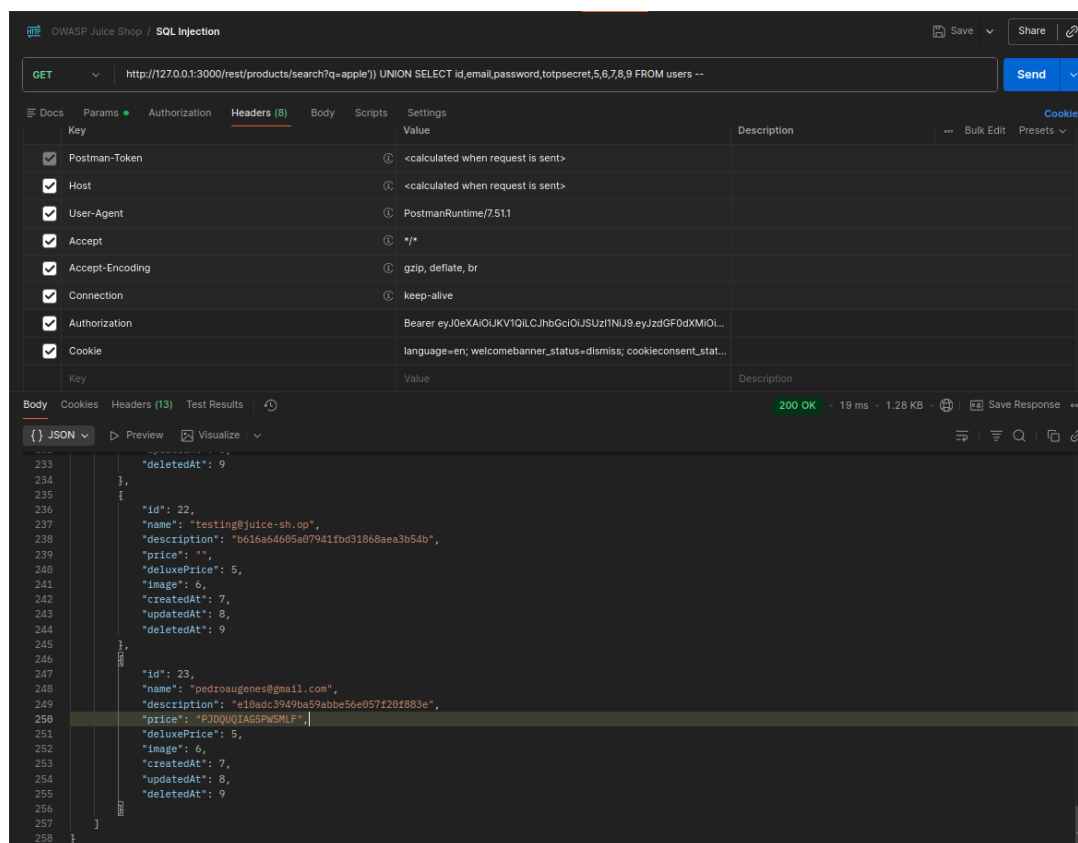


Figure 3.3: Postman request demonstrating the leakage of the TOTP secret from the database if a user has 2FA enabled. "price" field contains the TOTP secret in plaintext, which can be imported into a 2FA app to generate valid one-time passwords.

3.3.4 Remediation Recommendations

Recommended Fixes

1. Encrypt TOTP Secrets

Store TOTP secrets securely using strong encryption (e.g., AES with proper key management). Secrets must never be stored in plaintext.

3.4 JWT Token Forgery

Vulnerability Overview

Vulnerability Class	Vulnerable Component
Location	JW Token, Server
CVSS v4 Score	9.3 (Critical)
MITRE ATT&CK	Tactic: Credential Access ID: T1606 (Forge Web Credentials)

3.4.1 Root Cause

The application uses JSON Web Tokens (JWT) for authentication, but the implementation is flawed. The server does not properly verify the signature of the JWT tokens, allowing attackers to forge tokens with arbitrary payloads. By crafting a JWT with a manipulated payload (e.g., changing the `userId` to 1), an attacker can gain unauthorized access to the system as if they were the user with that ID.

3.4.2 Impact Analysis

This vulnerability has a critical impact on the security of the application, as it allows attackers to bypass authentication and gain unauthorized access to user accounts and sensitive data.

Security Property	Impact
Confidentiality	Attackers can access sensitive user data by forging JWT tokens.
Integrity	Attackers can impersonate users and perform unauthorized actions.
Availability	Compromised accounts may lead to abuse and service disruption.

3.4.3 Evidence

JWT tokens were successfully forged by manipulating the payload and bypassing signature verification, granting unauthorized access to user accounts.

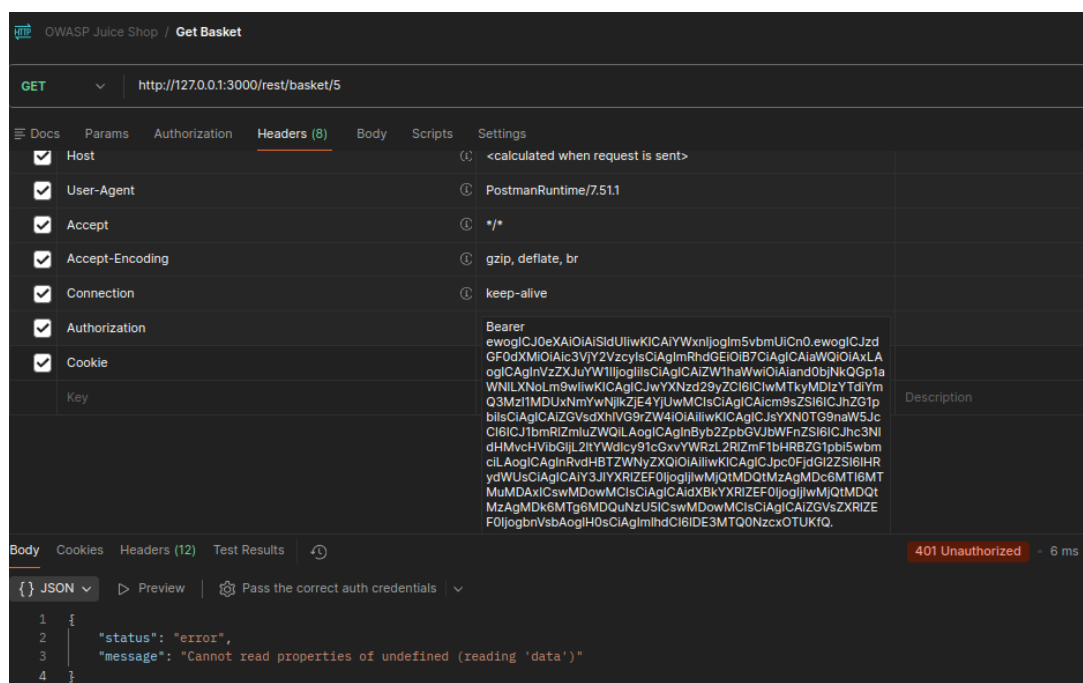


Figura 3.4: Postman request demonstrating the forging of a JWT token to gain unauthorized access to a user account.

3.4.4 Remediation Recommendations

Recommended Fixes

1. Enforce JWT Signature Verification (Critical)

Ensure that the server strictly verifies JWT signatures and rejects insecure configurations such as tokens using "**alg**": "**none**" and untrusted or unspecified algorithms

Explicitly whitelist secure algorithms (e.g., HS256, RS256):

Never rely on the algorithm specified in the token header.

2. Use Strong Signing Keys

Use cryptographically strong keys for signing tokens

3. Do Not Trust Token Claims Alone

Always validate token contents against the server-side state:

- Verify that `userId` exists
- Ensure the user account is active
- Validate authorization roles and permissions

JWTs should be treated as input, not as a source of truth.

4. Remove Sensitive Data from JWT Payload

Avoid including sensitive information inside JWTs. Password hashes, TOTP secrets, or any personally identifiable information (PII) should never be part of the token payload.

5. Implement Token Integrity & Validation Checks

Validate standard JWT claims to ensure token integrity with checks such as `exp` (expiration), `nbf` (not before), and `aud` (audience) to prevent replay attacks and ensure tokens are used within their intended context.

Ensure tokens are rejected if any of these validations fail.

Capítulo 4

Phase B Findings

4.1 Attack Plan

Vulnerability Class	WSTG Test Case	Purpose
Brute Force	WSTG-ATHN-07	Assess whether weak or predictable credentials can be discovered through password guessing or dictionary-based attacks.
Cross-Site Request Forgery (CSRF)	WSTG-SESS-05	Determine whether sensitive actions can be triggered without proper anti-CSRF protection.
Insecure CAPTCHA	WSTG-ATHN-03	Assess whether CAPTCHA controls can be bypassed, reused, or manipulated to permit automated or unauthorised actions.

Tabela 4.1: Planned vulnerability classes and associated WSTG test cases

4.1.1 Brute Force Testing Plan

The first vulnerability class to be tested is Brute Force. The relevant WSTG test case is **WSTG-ATHN-07: Testing for Weak Password Policy**. The objective is to determine whether the DVWA login mechanism allows weak credentials to be guessed using a controlled dictionary or traditional brute-force approach.

The hypothesis is that the Medium-level control can be bypassed by carefully analysing HTTP responses and automating requests at a controlled rate. Burp Suite may be used to intercept login attempts and compare differences between failed and successful responses. A small dictionary attack may be performed against known DVWA users, such as the default administrative account, while monitoring response content, status codes, redirects, and timing behaviour.

4.1.2 Cross-Site Request Forgery Testing Plan

The second vulnerability class to be tested is Cross-Site Request Forgery. The relevant WSTG test case is **WSTG-SESS-05: Testing for Cross Site Request Forgery**. The objective is to determine whether DVWA allows an attacker to force an authenticated user to perform a sensitive action without the user's explicit consent.

The hypothesis is that the Medium-level control can be bypassed if the protection relies only on weak request validation, such as predictable request structure or insufficient referrer checking. Testing will involve capturing a legitimate password-change request, identifying whether an anti-CSRF token is present, and attempting to replay or reproduce the request from an external page or crafted link.

4.1.3 Insecure CAPTCHA Testing Plan

The third vulnerability class to be tested is Insecure CAPTCHA. The relevant test cases are **WSTG-ATHN-03: Testing for Weak Lock Out Mechanism** and **OWASP-AT-012: Testing for CAPTCHA**. The objective is to determine whether the CAPTCHA mechanism prevents automated or unauthorised actions, or whether it can be bypassed through request manipulation.

The hypothesis is that the Medium-level CAPTCHA control may be bypassed by intercepting and modifying requests in Burp Suite. Testing will focus on whether CAPTCHA validation is performed securely on the server, whether successful CAPTCHA responses can be reused, and whether sensitive actions can be submitted directly without completing the CAPTCHA challenge.

4.2 Summary of Findings

ID	Finding Title	Vulnerability Class	Severity
B-01	[Finding title]	[Class]	[High]
B-02	[Finding title]	[Class]	[Critical]
B-03	[Finding title]	[Class]	[Low]
B-04	[Finding title]	[Class]	[Medium]

Tabela 4.2: Summary of Phase B findings

4.3 Finding B-01: [Finding Title]

Vulnerability Class

[Example: Brute Force / CSRF / File Inclusion / File Upload / Command Injection / SQL Injection / XSS]

Affected DVWA Module

- **Module:** [DVWA module name]
- **Security Level:** Medium
- **Target URL:** http://[DVWA-IP]/dvwa/vulnerabilities/[module]/

Description

[Briefly describe the vulnerability. Explain what the weakness is, where it appears in DVWA, and why it matters.]

Medium-Security Control Observed

[Describe the defence DVWA applies at Medium security level. For example: input filtering, request validation, delay between login attempts, file extension checks, referrer checks, or command character filtering.]

Bypass Hypothesis

[Explain how you believed the Medium-level control could be bypassed before testing.]

Evidence

Request

[Paste intercepted HTTP request, curl [command](#), payload, or tool output here]

Listing 4.1: HTTP request or command used for Finding B-01

Response

[Paste relevant response body, status code, output, or error message here]

Listing 4.2: Relevant response evidence for Finding B-01

Screenshot Evidence

Figura 4.1: Evidence for Finding B-01

Steps to Reproduce

1. Log in to DVWA using a valid test account.
2. Set the DVWA security level to *Medium*.
3. Navigate to [target module].
4. Submit the following payload or request:
[Payload or [command](#)]
5. Observe that [describe result].

Impact

[Explain what an attacker could achieve. Mention confidentiality, integrity, availability, authentication bypass, account compromise, file disclosure, command execution, or privilege abuse where relevant.]

Remediation

Remediation item 1

Remediation item 2

Remediation item 3

Severity Justification

Severity: [Low/Medium/High/Critical]

[Justify the severity based on exploitability, required privileges, attacker effort, and impact.]

4.4 Finding B-02: [Finding Title]

Vulnerability Class

[Insert vulnerability class]

Affected DVWA Module

- **Module:** [DVWA module name]
- **Security Level:** Medium
- **Target URL:** http://[DVWA-IP]/dvwa/vulnerabilities/[module]/

Description

[Describe the weakness and the affected functionality.]

Medium-Security Control Observed

[Describe what DVWA attempts to do at Medium level.]

Bypass Hypothesis

[Explain the intended bypass or testing logic.]

Evidence

Request

[Insert request, payload, or [command](#)]

Listing 4.3: HTTP request or command used for Finding B-02

Response

[Insert response evidence]

Listing 4.4: Relevant response evidence for Finding B-02

Screenshot Evidence

Figura 4.2: Evidence for Finding B-02

Steps to Reproduce

Step 1

Step 2

Step 3

Step 4

Impact

[Explain the security consequence.]

Remediation

Remediation item 1

Remediation item 2

Remediation item 3

Severity Justification

Severity: [Low/Medium/High/Critical]

[Explain why this severity level was selected.]

4.5 Finding B-03: [Finding Title]

Vulnerability Class

[Insert vulnerability class]

Affected DVWA Module

- **Module:** [DVWA module name]
- **Security Level:** Medium
- **Target URL:** http://[DVWA-IP]/dvwa/vulnerabilities/[module]/

Description

[Describe the finding.]

Medium-Security Control Observed

[Describe the Medium-level control.]

Bypass Hypothesis

[Explain the bypass idea.]

Evidence

Request

[Insert request, payload, or [command](#)]

Listing 4.5: HTTP request or command used for Finding B-03

Response

[Insert response evidence]

Listing 4.6: Relevant response evidence for Finding B-03

Screenshot Evidence

Figura 4.3: Evidence for Finding B-03

Steps to Reproduce

Step 1

Step 2

Step 3

Step 4

Impact

[Explain what this finding allows an attacker to do.]

Remediation

Remediation item 1

Remediation item 2

Remediation item 3

Severity Justification

Severity: [Low/Medium/High/Critical]
[Justify severity.]

4.6 Finding B-04: [Finding Title]

Vulnerability Class

[Insert vulnerability class]

Affected DVWA Module

- **Module:** [DVWA module name]
- **Security Level:** Medium
- **Target URL:** http://[DVWA-IP]/dvwa/vulnerabilities/[module]/

Description

[Describe the vulnerability and why it is relevant.]

Medium-Security Control Observed

[Describe the control implemented by DVWA Medium.]

Bypass Hypothesis

[Explain how the Medium control may be bypassed.]

Evidence

Request

[Insert request, payload, or [command](#)]

Listing 4.7: HTTP request or command used for Finding B-04

Response

[Insert response evidence]

Listing 4.8: Relevant response evidence for Finding B-04

Screenshot Evidence

Figura 4.4: Evidence for Finding B-04

Steps to Reproduce

Step 1

Step 2

Step 3

Step 4

Impact

[Explain the practical impact of the issue.]

Remediation

Remediation item 1

Remediation item 2

Remediation item 3

Severity Justification

Severity: [Low/Medium/High/Critical]
[Explain severity decision.]

4.7 Constraint Coverage Matrix

Requirement	How It Is Satisfied	Status
At least four distinct findings	Findings B-01, B-02, B-03, and B-04 are documented.	[Met/Not Met]
At least three vulnerability classes	Findings cover: [Class 1], [Class 2], and [Class 3].	[Met/Not Met]
At least one new Phase B class	Finding [B-XX] covers [File Inclusion/File Upload/Command Injection].	[Met/Not Met]
At least one Medium-control bypass	Finding [B-XX] demonstrates bypass of [specific Medium control].	[Met/Not Met]
Same evidence standard as Phase A	Each finding includes request, response, screenshots, reproduction steps, impact, and remediation.	[Met/Not Met]

Tabela 4.3: Phase B assessment constraint coverage matrix

4.8 Phase B Conclusion

The Phase B assessment identified four distinct vulnerabilities across at least three vulnerability classes. The results demonstrate that DVWA Medium-level controls increase attack difficulty but may still be bypassed when the attacker understands the defensive logic in place. The findings show weaknesses in authentication, session management, and input-handling controls, including at least one vulnerability class not assessed during Phase A.

Overall, the assessment confirms that partial input validation or client-side/request-level checks are insufficient when not supported by strong server-side validation, secure session handling, and robust access control mechanisms.

Capítulo 5

Cross-Phase Analysis

Technique transfer, adaptation, and reflection on DVWA's medium-security controls, as specified in §6.3 above.

Capítulo 6

Conclusion

Prioritised remediation recommendations for both targets. Overall assessment of engagement quality and lessons learned

Capítulo 7

Appendix A - Evidence

7.1 SQL Injection in /rest/products/search

This appendix section contains the screenshots and supporting material referenced in Section ??.

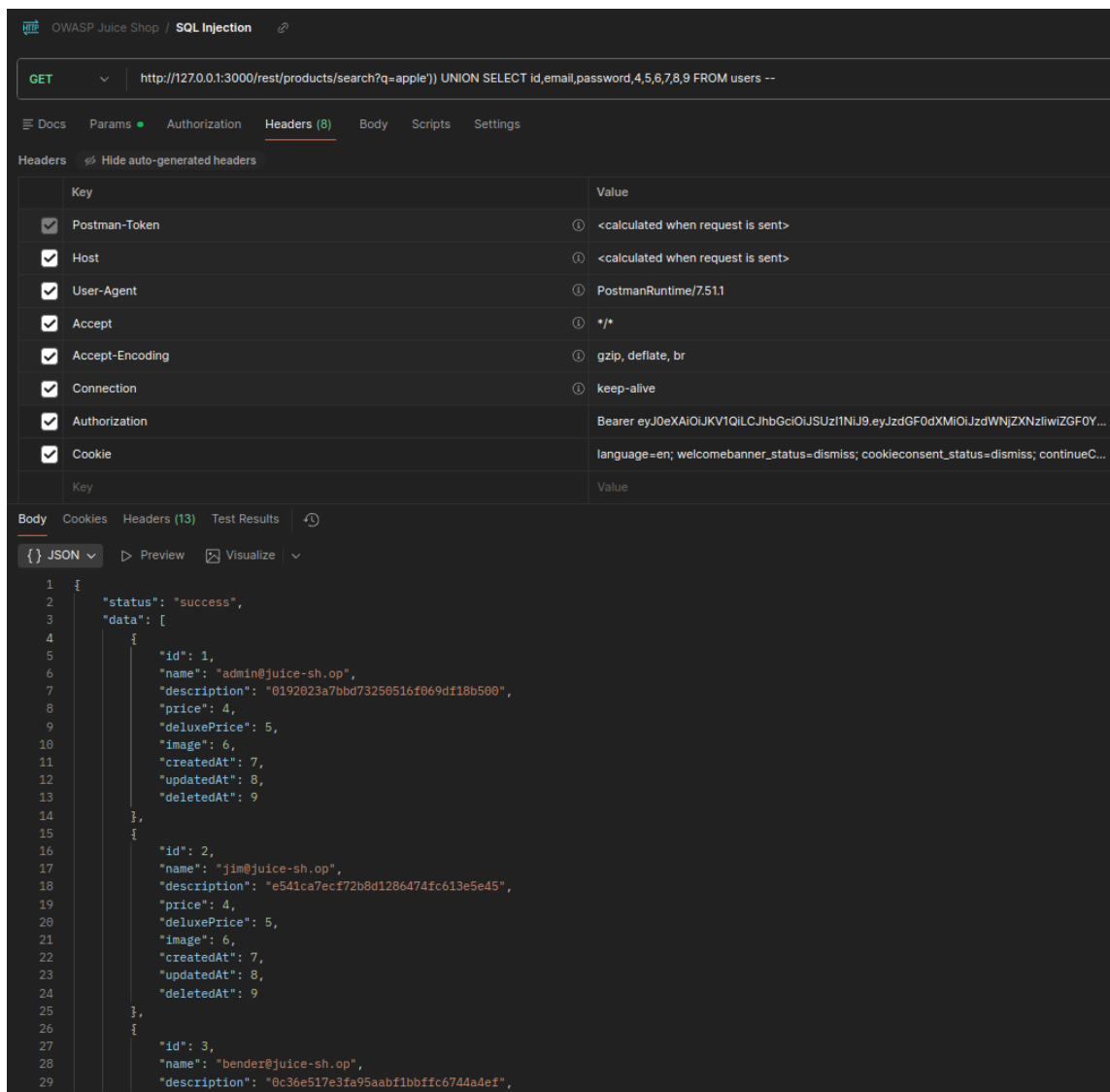


Figura 7.1: SQL injection payload submitted to the /rest/products/search endpoint.

Capítulo 8

Appendix B - Structured Event Log

Complete event log of Phase B exploitation activity, in the format specified by the instructor